

AI 510 Artificial Intelligence of Cloud Computing HOS01A-4 Introduction to Python Web Application Back-end

08/13/2022 Developed by Yared Shewarade

12/23/2022 Reviewed by Alekhya Malla

5/5/2024 Reviewed by Sam Chung

06/06/2024 Reviewed by Naveena Moddu

06/12/2024 Reviewed by Anh Nguyen

School of Technology and Computing (STC) @City University of Seattle (CityU)

Before You Start

- The directory path shown in the screenshots may be different from yours.
- Some steps are not explained in the tutorial. If you are not sure what to do:
 - Consult the resources listed below.
 - If you cannot solve the problem after a few tries, please contact the student worker through the MS Teams course channel.

Learning Outcomes

Students will be able to learn:

- Introduction to web application backend
- Setting up Virtual Environments
- Flask Installation
- Flask Application
- Flask - HTTP

Resource

- Flask, (n.d), Flask Documentation, <https://flask.palletsprojects.com/en/3.0.x/quickstart/>
- Tutorialspoint, (n.d), Flask Tutorial, <https://www.tutorialspoint.com/flask/index.htm>

Introduction to web application backend

A typical web application is divided into frontend and backend. Frontend returns a graphical representation of contents to the user while the backend provides necessary computed results back to the frontend.

For intermediate backend testing, the frontend is usually neglected and shows string results to a web browser. In this module, we will practice making a backend using Python web application libraries such as Flask or FastAPI.

Flask is a web application framework written in Python that provides tools, libraries, and technologies to build a web application. It is a micro framework with little to no dependencies, and it has two dependencies:

- Werkzeug -> WSGI utility library
- Jinja2 -> template engine for Python

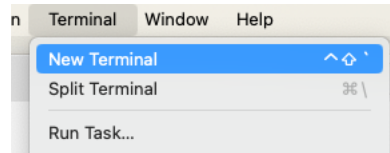
Web Server Gateway Interface (WSGI) is a Python protocol for a universal interface between the web server and the web applications.

Virtual Environment setup

Virtual environment is a good programming practice for managing the dependencies for your project, both in development and in production. Virtual environments are independent groups of Python libraries, one for each project. Installed packages for one project will not affect other projects or the operating system's packages.

Step 1) Python 3 virtual environment can be used to isolate libraries used for specific projects. Let's install virtualenv.

Open VSCode, New Terminal and type “`pip install virtualenv`”



```
(base) yaredshewarade@yared ~ % pip install virtualenv
Collecting virtualenv
  Downloading virtualenv-20.16.3-py2.py3-none-any.whl (8.8 MB)
    | 8.8 MB 3.4 MB/s
Requirement already satisfied: filelock<4,>=3.4.1 in ./opt/anaconda3/lib/python3.9/site-packages (from virtualenv) (3.6.0)
Requirement already satisfied: platformdirs<3,>=2.4 in ./opt/anaconda3/lib/python3.9/site-packages (from virtualenv) (2.4.0)
Collecting distlib<1,>=0.3.5
  Downloading distlib-0.3.5-py2.py3-none-any.whl (466 kB)
    | 466 kB 15.6 MB/s
Installing collected packages: distlib, virtualenv
Successfully installed distlib-0.3.5 virtualenv-20.16.3
(base) yaredshewarade@yared ~ %
```

Step 2) Create a folder in your working folder (ex. Desktop/AI510/HOS/AI510-Summer2024-HOS01/Module01)

Note: You can use any folder name. “Flask_Project” is used for this practice.

Type the following code on the VS Code Terminal

- `mkdir Flask_Project` #Creating folder
- `cd Flask_Project` #Changing working directory
- From top menu “Terminal” > “New Terminal”
- `virtualenv venv` OR `python -m venv venv` # Creating virtual environment

Note: For Windows Users, you can use this `python -m venv venv` to create a virtual environment

```
(base) yaredshewarade@yared Flask_H0S01 % mkdir Flask_Project
(base) yaredshewarade@yared Flask_H0S01 % cd Flask_Project
(base) yaredshewarade@yared Flask_Project % virtualenv venv
created virtual environment CPython3.9.12.final.0-64 in 255ms
creator CPython3Posix(dest=/Users/yaredshewarade/Desktop/City University/TA_Job/
Classroom/AI 510 - Artificial Intelligence in CCloud Computing/Flask_H0S01/Flask_Pr
oject/venv, clear=False, no_vcs_ignore=False, global=False)
seeder FromAppData(download=False, pip=bundle, setuptools=bundle, wheel=bundle,
via=copy, app_data_dir=/Users/yaredshewarade/Library/Application Support/virtualen
v)
added seed packages: pip==22.2.2, setuptools==63.4.1, wheel==0.37.1
activators BashActivator,CShellActivator,FishActivator,NushellActivator,PowerShe
llActivator,PythonActivator
(base) yaredshewarade@yared Flask_Project %
```

Step 3) Activate the virtual environment.

Note: The virtual environment should be activated before working on the flask project. Type the following code to activate the environment in the terminal.

- For OSX

```
venv/bin/activate
```

In case you face the permission denied issue while using the above command. Please run the below commands. Below are images showing the above command and ways to handle the permission denied issue by using any of the below commands.

```
. venv/bin/activate
```

(Note: space between dot and 'venv/bin/activate')

```
[(venv) (base) vivekpottam@Alekhya-MacBook-Pro Flask_Project % venv/bin/activate
zsh: permission denied: venv/bin/activate
[(venv) (base) vivekpottam@Alekhya-MacBook-Pro Flask_Project % . venv/bin/activate
```

(Or)

```
source venv/bin/activate
```

```
[(venv) (base) vivekpottam@Alekhya-MacBook-Pro Flask_Project % venv/bin/activate
zsh: permission denied: venv/bin/activate
[(venv) (base) vivekpottam@Alekhya-MacBook-Pro Flask_Project % source venv/bin/activate
```

- For Windows

```
venv\Scripts\activate
```

```
PS C:\Users\Administrator\Desktop\TA\Summer\AI510\flask_project> python -m venv venv
PS C:\Users\Administrator\Desktop\TA\Summer\AI510\flask_project> venv\Scripts\activate
(venv) PS C:\Users\Administrator\Desktop\TA\Summer\AI510\flask_project>
```

Note: Virtual environment can be deactivated using `deactivate`

Flask Installation

Type the following code to install Flask.

```
pip install Flask
```

Note: Virtual environment is not needed to install Flask. It can be installed on the system without venv on the base Python installation.

```
(venv) (base) hongeinh@hongeinh-Lenovo-IdeaPad-S540-15IML:~/Documents/School/CityU/Assistant/2023-2024/Summer
• /AI510/01/Flask_Project$ pip install Flask
Collecting Flask
  Downloading Flask-2.2.5-py3-none-any.whl.metadata (3.9 kB)
Collecting Werkzeug>=2.2.2 (from Flask)
  Downloading Werkzeug-2.2.3-py3-none-any.whl.metadata (4.4 kB)
Collecting Jinja2>=3.0 (from Flask)
  Downloading Jinja2-3.1.4-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.0 (from Flask)
  Downloading itsdangerous-2.1.2-py3-none-any.whl.metadata (2.9 kB)
Collecting click>=8.0 (from Flask)
  Downloading click-8.1.7-py3-none-any.whl.metadata (3.0 kB)
Collecting importlib-metadata>=3.6.0 (from Flask)
  Downloading importlib-metadata-6.7.0-py3-none-any.whl.metadata (4.9 kB)
Collecting zipp>=0.5 (from importlib-metadata>=3.6.0->Flask)
  Downloading zipp-3.15.0-py3-none-any.whl.metadata (3.7 kB)
Collecting typing-extensions>=3.6.4 (from importlib-metadata>=3.6.0->Flask)
  Downloading typing_extensions-4.7.1-py3-none-any.whl.metadata (3.1 kB)
Collecting MarkupSafe>=2.0 (from Jinja2>=3.0->Flask)
  Downloading MarkupSafe-2.1.5-cp37-cp37m-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (3.0 kB)
Downloading Flask-2.2.5-py3-none-any.whl (101 kB)
101.8/101.8 kB 2.9 MB/s eta 0:00:00
Downloading click-8.1.7-py3-none-any.whl (97 kB)
97.9/97.9 kB 15.9 MB/s eta 0:00:00
Downloading importlib-metadata-6.7.0-py3-none-any.whl (22 kB)
Downloading itsdangerous-2.1.2-py3-none-any.whl (15 kB)
Downloading Jinja2-3.1.4-py3-none-any.whl (133 kB)
133.3/133.3 kB 16.3 MB/s eta 0:00:00
Downloading Werkzeug-2.2.3-py3-none-any.whl (233 kB)
233.6/233.6 kB 24.6 MB/s eta 0:00:00
Downloading MarkupSafe-2.1.5-cp37-cp37m-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (25 kB)
Downloading typing_extensions-4.7.1-py3-none-any.whl (33 kB)
Downloading zipp-3.15.0-py3-none-any.whl (6.8 kB)
Installing collected packages: zipp, typing-extensions, MarkupSafe, itsdangerous, Werkzeug, Jinja2, importlib-
-metadata, click, Flask
Successfully installed Flask-2.2.5 Jinja2-3.1.4 MarkupSafe-2.1.5 Werkzeug-2.2.3 click-8.1.7 importlib-metadat
a-6.7.0 itsdangerous-2.1.2 typing-extensions-4.7.1 zipp-3.15.0
```

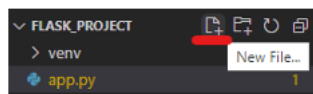
base is changed to venv

Flask Application

Now we have created a virtual environment and installed a flask, we can begin to write code.

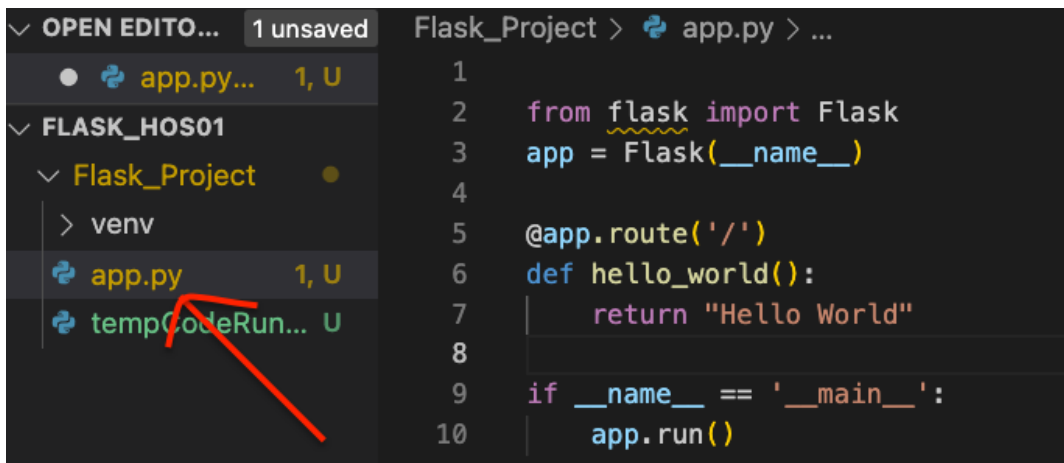
Below is the system to run a simple web application that says Hello World.

Add a blank “app.py” file by clicking “New File” and giving the name “app.py”.



“app.py” should lie directly under the **Flask_Project**, not under the venv folder.

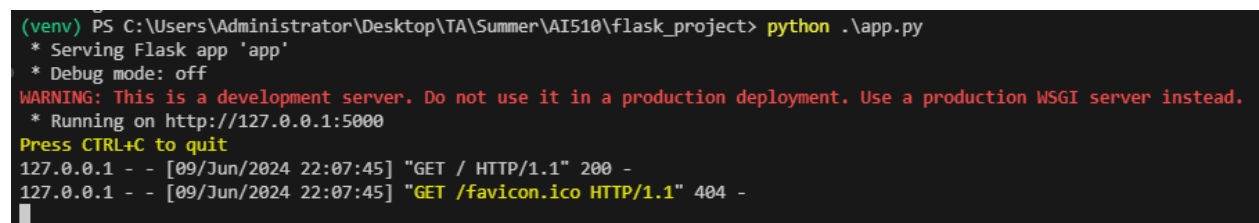
Type the following code in your project **Flask_Project** folder.



```
1
2 from flask import Flask
3 app = Flask(__name__)
4
5 @app.route('/')
6 def hello_world():
7     return "Hello World"
8
9 if __name__ == '__main__':
10    app.run()
```

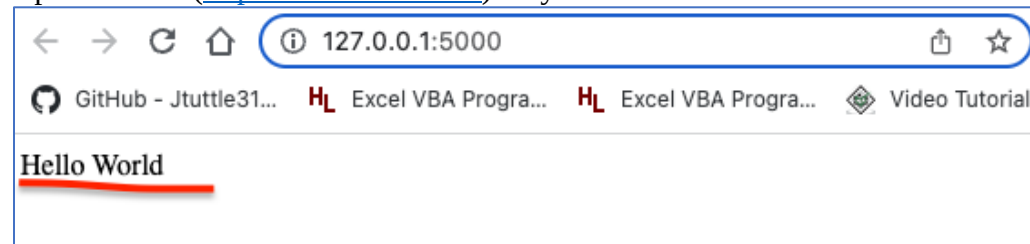
Execute the Python script with `python {script_name}.py`. For example, if your script name is `app.py`, you would type the following to run the script.

```
python app.py
```



```
(venv) PS C:\Users\Administrator\Desktop\TA\Summer\AI510\flask_project> python .\app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [09/Jun/2024 22:07:45] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [09/Jun/2024 22:07:45] "GET /favicon.ico HTTP/1.1" 404 -
```

Open the link (<http://127.0.0.1:5000/>) on your browser and it should look like this:



Take a screenshot of your flask application(`app.py`) terminal output as well as web browser after opening the local server backend page and save it to your working folder.

Let's understand the syntax of the program:

1. The first line of code `"from flask import Flask"` is importing Flask package in the file.
2. The second line of the code creates an object of the flask class using `"app = flask()"`
3. Flask constructor takes the name of the current module(`__name__`) as an argument and to default constructor which will help Flask to look at templates and static files.
4. `route( )` function is a decorator that tells the application which URL should call the associated function.
5. The route decorator helps to define which routes should be navigated to the following function. `"@app.route(/)"`.
6. `'/'` let Flask direct all the default root path traffic to the defined root path function.
7. We define the function and the action to be performed.

8. '/' URL is bound with the `hello_world()` function, and the output of this function will be rendered when the home page of the web server is opened in the browser.
9. The default content type is HTML, so our string will be wrapped in HTML and displayed in the browser.
10. In the main function created, '`app.run()`' will initiate the server to run on the local development server.

Routes:

There are some basic functionalities about routes that should be discussed to construct a basic application. The '`route()`' as noted earlier is a Flask decorator.

`app.route(rule, options)`

- The `rule` parameter represents URL binding with the function.
- The `options` are a list of parameters to be forwarded to the underlying Rule object.

Debugging:

We should add debugging to a Flask application for two main reasons:

- Errors will be traced and displayed.
- Changes reflect instantaneously when there is a change is made to the application.

Debug can be added using the following code:

```
8 if __name__ == '__main__':
9     app.run(debug = True)
```

Flask – HTTP Methods

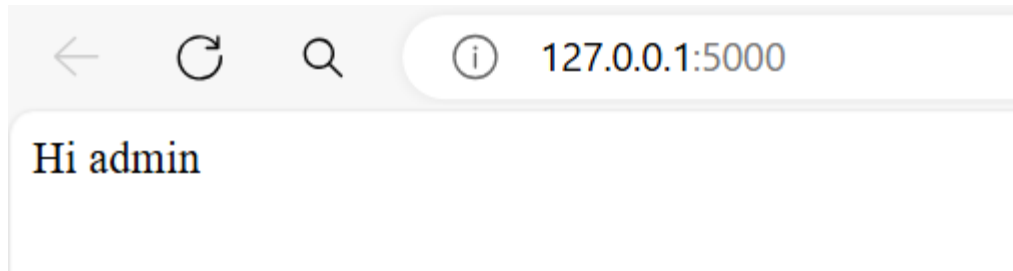
The Hypertext Transfer Protocol (HTTP) protocol is the foundation of data communication on the World Wide Web. It is a protocol that allows clients and servers to share information. HTTP is a request-response protocol used to communicate between a client and a server. A client (browser) sends an HTTP request to the server and the server answers.

HTTP methods:

- GET -> used to fetch information from a certain website.
- POST -> used to send data to a server to update or create a resource.
- PUT -> Replaces all current representations of the target resource with the uploaded content.
- HEAD -> Same as GET without the response body.
- DELETE -> Removes all current representations of the target resource with the uploaded content.

Note: GET and POST are the most used methods.

Write a Flask application with your name. The browser output will be as follows:



HOS submission instructions:

1. Please install the GitHub Desktop: https://cityuseattle.github.io/docs/git/github_desktop/
2. Clone, organize, and submit your work through GitHub Desktop: <https://cityuseattle.github.io/docs/hoporhos>